

Spec driven development II

Plan to convergence

Plan:

- Module I (29/9):
 - Session I: What is AI? What are AI agents?
 - Session II: Coding agents and spec driven development
 - Session III: Case – Constitution, specify and clarify with AI
- Module II (6/10):
 - Plan and task part of spec driven development
 - Code clickable POC with coding agent (case)
 - Evaluate POC (case)

The workflow

Constitution → Specify → Clarify → Plan → Tasks → Implement → Converge

- **Constitution:** agree on project-wide principles and constraints.
- **Specify:** describe the feature, users, requirements, and success criteria.
- **Clarify:** find and resolve gaps, ambiguity, and unanswered questions.
- **Plan:** choose the technical approach and design.
- **Tasks:** break the work into implementable steps.
- **Implement:** build and test the selected work.
- **Converge:** check what is complete and identify remaining gaps.

Checklists and Analyze provide extra quality and consistency checks along the way.

Plan - from specification to technical plan

- Converts your specification into a high-level technical implementation plan
- Spec said **what** — plan says **how**: language/stack (Python today), architecture, main components, data model
- Output: a plan.md the agent works from — not code yet, just the approach

Tasks – break the plan into user stories

- Breaks the plan into small, concrete, actionable tasks
- Organized by **user stories** — each user story becomes one or more tasks, dependencies mapped
- Output: a tasks.md — a checklist the agent implements one item at a time

Implement - pick one story and build it for real

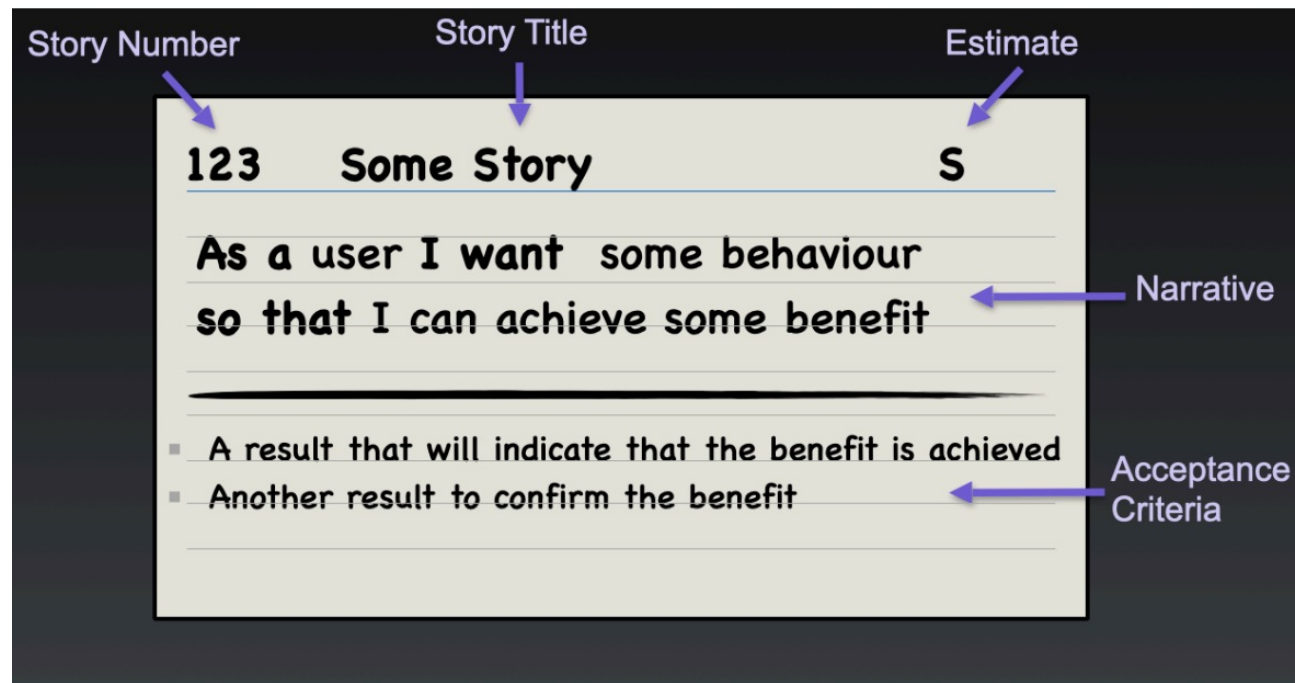
- Normally: work through every task until the full plan is built
- Today, **pick one task/user story only** — we don't have time for the whole system
- Goal: a small, clickable proof-of-concept slice, not a finished product
- Let the code agent do the actual coding — you review what it produces

Converge - does the code match the plan?

- Checks the finished code against your spec, plan, and tasks
- You can use your AI agent for this besides your own review
- It doesn't edit code itself — if it finds gaps, it appends new tasks instead
- Normal use: implement → converge → implement → converge, until it reports "Converged"
- For today, this is your evaluation step: does the POC actually do what the user story said? What's missing, wrong, or different from what you planned?

User stories

User stories



User stories

- They are not really requirements especially not as use cases can be
- Use cases are (mostly) written by the customer
- They have enough detail to enable developers to understand the functionality and make an estimate. No more.
- There must be an acceptance criteria to test that the functionality is implemented.

What is a user story?

- a user story is an informal, natural language description of features of a software system.
- They are written from the perspective of an end user or user of a system, and may be recorded on index cards, Post-it notes, or digitally in specific management software.
- Depending on the product, user stories may be written by different stakeholders like client, user, manager, or development team

User story templates

- As a <user>, I can <some goal>, so that <some reason/benefit>
- Describes
 - Who
 - Their intent
 - The result or reason for the intent

Gode stories

- Beskriver præcist og simpelt HVAD brugeren ønsker software skal gøre.
- Inkluderer ikke specifikke detaljer om HVORDAN software skal virke
- Beskrives i naturligt sprog , som ikke-tekniske personer og brugeren kan forstå.
- Det er en grænseflade mellem bruger og udviklings-teamet

The Three Cs

- 1.Card:** Written description of the story, used for planning and as a reminder
- 2.Conversation:** Conversations about the story that serve to flesh out the details of the story
- 3.Confirmation:** Tests that convey and document details that can be used to determine when a story is complete.

Examples

- As a user, I can indicate folders not to backup so that my backup drive isn't filled up with things I don't need saved.
- As a power user, I can specify files or folders to backup based on file size, date created and date modified.

Template

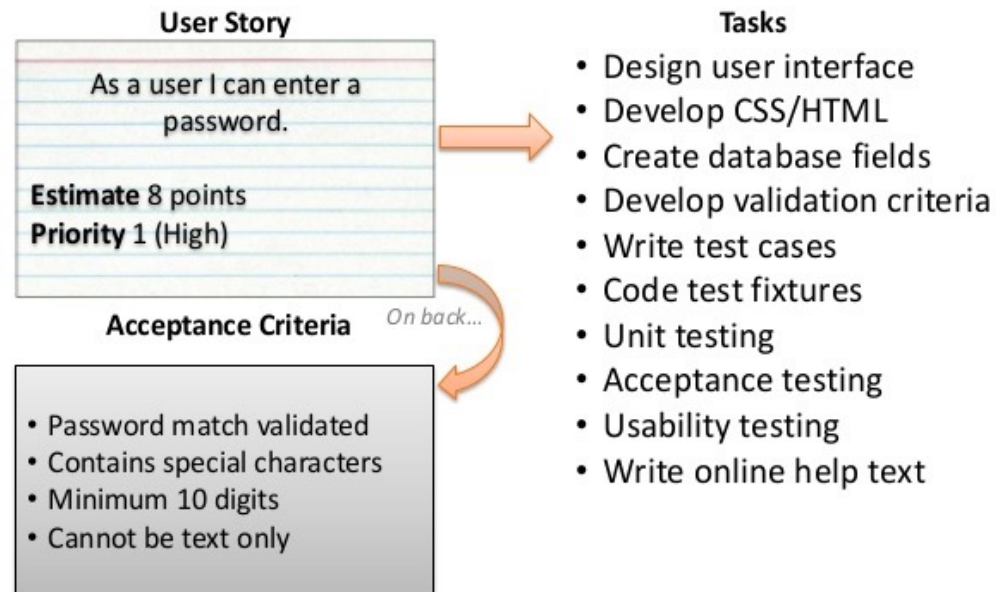
Story Number	Story Title	Estimate
123	Some Story	S
As a user I want some behaviour so that I can achieve some benefit		
▪ A result that will indicate that the benefit is achieved ▪ Another result to confirm the benefit		

Narrative

Acceptance Criteria

Decomposition of User Stories

From User Stories to **Tasks**



INVEST principle

- “I” ndependent (of all others)
- “N” egotiable (not a specific contract for features)
- “V” aluable
- “E” stimable (to a good approximation)
- “S” mall (so as to fit within an iteration)
- “T” estable (in principle, even if there isn’t a test for it yet)